

Session 02

payments-platform - Arihant Rakhecha

15 September 2026

- [How to read this](#)
- [Part 1 – What this session accomplished](#)
- [Part 2 – New vocabulary from this session](#)
 - [Exit codes](#)
 - [HTTP status codes as a contract](#)
 - [Heredoc](#)
 - [Shell chaining: `&&` versus separate lines](#)
 - [Amending a commit](#)
- [Part 3 – The five environment problems](#)
 - [Problem 1 – `make topics` failed: container is not running](#)
 - [Problem 2 – `zsh: command not found: ruff`](#)
 - [Problem 3 – Kafka: `advertised.listeners cannot use 0.0.0.0`](#)
 - [Problem 4 – recreating one container without destroying the database](#)
 - [Problem 5 – the TOML append that broke the config](#)
- [Part 4 – The two real bugs](#)
 - [Bug 1 – 500 where 409 was correct](#)
 - [Bug 2 – a demo that validated nothing](#)
 - [What these two bugs have in common](#)
- [Part 5 – Consumer groups and rebalancing](#)
- [Part 6 – Claude Code, and reviewing a diff](#)
 - [The OAuth paste failure](#)
 - [Why approve edits one at a time](#)
 - [What a good diff review looks like](#)
- [Part 7 – Every command from this session](#)
 - [Diagnostic sequence worth memorising](#)
- [Part 8 – Where you actually are](#)
 - [Commits from this session](#)
 - [Next: milestone 3](#)
- [Glossary – cumulative](#)

How to read this

Session 01 ended with three containers started. This picks up exactly there and runs to a working, verified milestone 2.

Session 01 was mostly *setup* problems – things not installed, wrong versions, files in the wrong place. This session is mostly *logic* problems, which are more interesting, because two of them were genuine bugs in the project code that the test suite could not see.

Same structure as session 01. The **“Why not the alternative”** notes are still the part that matters most.

Part 1 - What this session accomplished

At the start: three containers running, no database tables, no topics, nothing verified.

At the end: milestone 2 complete and proven end to end. A payment flows from an HTTP request, through Postgres, through the outbox, through Kafka, to a consumer – and every invariant holds.

Along the way, eight problems. Five were environment or tooling issues. **Three were real bugs**, and finding them was the most valuable part of the evening:

1. A Kafka config error that crashed the broker on startup
2. A status-code bug in the payments service – returning 500 where 409 was correct
3. A demo script that silently validated nothing on any run after the first

The last two are worth dwelling on, because **neither was catchable by the 17 unit tests**, and both belong to the same category.

Part 2 - New vocabulary from this session

Exit codes

When a program stops, it reports a number. `0` means success. Anything else is failure, and the specific number is a clue.

`docker compose ps -a` showed:

```
pp-kafka    Exited (1)
```

`1` means the program itself decided to stop – it hit a condition it refused to continue past. That points at configuration or bad input.

`137` would have meant something entirely different: the program was *killed from outside* for using too much memory. On an 8 GB laptop with a Docker memory limit, that was a genuine possibility worth ruling out.

Seeing `(1)` rather than `(137)` immediately eliminated “out of memory” and pointed at config. That one number saved going down the wrong path.

Why `-a` matters. Plain `docker compose ps` lists only *running* containers, so a crashed container is simply absent – and “absent” looks the same as “never started”. `-a` means “all”, including stopped ones, with their exit codes. **When something is missing, ask to see the dead ones.**

HTTP status codes as a contract

A status code is not decoration. It is a machine-readable instruction telling the client what to do next.

Code	Means	What a client should do
<code>200 OK</code>	Worked	Carry on
<code>201 Created</code>	Worked, something new exists	Carry on
<code>409 Conflict</code>	Valid request, but the resource is not in a state to allow it	Stop. Do not retry.
<code>422 Unprocessable</code>	Well-formed, but the world said no (e.g. insufficient funds)	Stop, tell the user
<code>500 Internal Server Error</code>	The server broke	Retry - it might work next time

The 409/500 distinction caused a real bug this session. A client receiving 500 for “this already happened” will retry forever, because 500 promises that retrying is reasonable.

Heredoc

The thing that made you ask “should I press Ctrl+C”.

```
python3 - <<'PY'
some code
PY
```

`<<'PY'` tells the shell: read everything that follows as input until you see a line containing exactly `PY`. That terminator is not optional. Without it the shell waits for more input **forever** – which looks identical to a program hanging.

How to tell them apart: look at the prompt. A heredoc waiting for input shows `heredoc>` or a bare `>` instead of your normal prompt. A hung program shows no prompt at all.

Fix: type the terminator (`PY`) on its own line and press Enter.

`PY` is arbitrary – `EOF` is the common convention. The quotes around `'PY'` matter: they stop the shell from interpreting `$variables` inside the block, which is what you want when passing code through verbatim.

Shell chaining: `&&` versus separate lines

This caused a commit to go through despite a failing lint check.

```
ruff check . && pytest -q
git add -A && git commit -m "... " && git push
```

`&&` means “run the next command **only if** the previous one succeeded”. So `ruff` failing stopped `pytest`. But the **second line is a separate command**, unaffected by the first line’s failure. The commit ran regardless.

To make a failing check actually block a commit, it must all be one chain:

```
ruff check . && pytest -q && git add -A && git commit -m "... " && git push
```

Related operators worth knowing:

- `;` – run the next command regardless of success or failure
- `||` – run the next command **only if** the previous one *failed*
- `&&` – run the next command only on success

Amending a commit

```
git commit --amend -m "better message"
git push --force-with-lease
```

`--amend` replaces the most recent commit instead of adding a new one. Used here because a commit had landed with the message `"..."`.

Why `--force-with-lease` and not `--force`? Amending rewrites history, so the remote refuses a normal push. Plain `--force` says “overwrite whatever is there” – which destroys anyone else’s work if they pushed in between. `--force-with-lease` first checks the remote is where you last saw it, and refuses if not. Same outcome on a solo repo, but it cannot silently destroy work.

When not to do this at all: only amend commits nobody has pulled. Rewriting shared history forces everyone else to repair their clones.

Part 3 - The five environment problems

Problem 1 - `make topics` failed: container is not running

Symptom:

```
Error response from daemon: container 631cd248... is not running
make: *** [topics] Error 1
```

The insight that mattered: in session 01, `make up` printed `Container pp-kafka Started`. That was *true* – Docker did start it. It then crashed seconds later, and nothing told us.

“Started” is not “running”, and “running” is not “ready”. These are three different states. This is why `docker compose ps` has a `healthy` column, driven by the `healthcheck` block in `docker-compose.yml` – a command the container runs repeatedly to prove it is actually able to serve requests, not merely alive.

Diagnostic sequence:

```
docker compose logs kafka | tail -40
docker compose ps -a
docker stats --no-stream
```

Three questions: what did it say before dying, what is its exit code, and is memory the constraint? The answers were: a validation error, `Exited (1)`, and no – Postgres and Redis together were using 35 MB of a 3.8 GB limit.

Problem 2 - `zsh: command not found: ruff`

Cause: a mistake in the project setup. `ruff` and `pytest` were referenced by the CI workflow and by `CLAUDE.md`, but never listed in `requirements.txt`. They had simply never been installed.

The fix, and why it took this shape:

```
cat > requirements-dev.txt <<'EOF'
-r requirements.txt
pytest==8.3.4
pytest-asyncio==0.25.0
ruff==0.8.4
EOF
pip install -r requirements-dev.txt
```

The first line, `-r requirements.txt`, includes the other file. So `requirements-dev.txt` means “everything for running, plus the tools for developing”.

Why a separate file instead of adding them to `requirements.txt`? Because a production server needs to *run* the application, not test it. Shipping a test framework and a linter into production adds download size, disk usage, and more installed code that could contain a vulnerability – for zero benefit. The split is standard practice: runtime dependencies and development dependencies are different sets with different audiences.

Why not just `pip install ruff pytest` and move on? It would have worked tonight and broken later. An installed package that no file records is invisible: your CI does not know about it, a fresh clone does not get it, and in three weeks nobody remembers why something fails on a new machine. **If a tool is needed, a file must say so.**

Problem 3 - Kafka: `advertised.listeners` cannot use 0.0.0.0

The most technically interesting configuration error of the session.

The error:

```
java.lang.IllegalArgumentException: requirement failed:
advertised.listeners cannot use the nonroutable meta-address 0.0.0.0.
Use a routable IP address.
```

What 0.0.0.0 means. It is a wildcard meaning “all network interfaces on this machine”. Perfectly valid for *binding* – “accept connections arriving on any interface”. Completely meaningless as an *address to hand out*, because a client told “connect to 0.0.0.0” has nowhere to go. It is not an address; it is a statement about listening.

The word **nonroutable** in the error is precise: you cannot route traffic to it.

Why this only affects Kafka and not Postgres. Recall from session 01 that Kafka does something unusual – when a client connects, Kafka replies with the address to use for *subsequent* connections. It must do this because a real cluster has many brokers and has to tell clients which one holds which partition.

Postgres never does this. It just serves the connection. So 0.0.0.0 in a Postgres config is fine and in a Kafka *advertised* listener is fatal.

Our config used the literal 0.0.0.0 in `KAFKA_LISTENERS`, and the image’s startup wrapper carried it into the advertised-listener validation.

The fix – omit the host entirely:

```
KAFKA_LISTENERS: PLAINTEXT://:9092,PLAINTEXT_HOST://:29092,CONTROLLER://:9093
```

An empty host before the colon binds to all interfaces *identically* to 0.0.0.0, but sidesteps the validation. This is the form the official Apache Kafka examples use.

Problem 4 – recreating one container without destroying the database

A decision worth understanding, because the obvious move was dangerous.

The instinct when a container misbehaves is to restart everything:

```
make down && make up
```

That would have wiped the database. `make down` runs `docker compose down -v`, and the `-v` deletes **volumes** – the storage that outlives containers. The Postgres data directory is a volume. Every table just created by `make migrate` and the clearing account from `make seed` would have been destroyed.

What we did instead:

```
docker compose up -d --force-recreate kafka
```

Naming `kafka` at the end scopes the action to that one service. `--force-recreate` destroys and rebuilds the container even though the compose file’s *other* settings are unchanged, which is necessary because environment variables are baked in at container creation – a plain `restart` would reuse the old, broken config.

Why is `-v` in `make down` at all, if it is dangerous? Because during development a genuinely clean slate is often what you want, and a `down` that leaves stale data behind causes its own confusing bugs. It is the right default for a dev Makefile – but it means `make down` is destructive, and that is worth knowing before you type it.

The transferable rule: scope the smallest action that can fix the problem. Restarting everything is a blunt instrument that sometimes costs you work.

Problem 5 – the TOML append that broke the config

What was attempted:

```
printf '\nasyncio_default_fixture_loop_scope = "function"\n' >> pyproject.toml
```

What broke:

```
TOML parse error at line 11, column 1
|
11 | [tool.ruff.lint]
|   ^^^^^^^^^^^^^^^^^
unknown field `asyncio_default_fixture_loop_scope`
```

Why. `>>` appends to the end of the **file**. But TOML, YAML and INI have no closing brackets – a section runs until the *next* section header begins. So a key appended at the end of the file belongs to whatever section came last, which here was `[tool.ruff.lint.flake8-bugbear]`. The setting was intended for `[tool.pytest.ini_options]`, several sections earlier.

The error message is worth reading closely, because it is subtly misleading. It points at **line 11**, `[tool.ruff.lint]` – but the problem is on the last line of the file. TOML parsers report the *section* that received the unknown key, not the line the key sits on.

Fix: rewrite the whole file with the key in the right section.

Why not use a tool to edit it? You could (`sed`, `yq`, `tomlkit`). For a 25-line file, rewriting it is faster, and you see the result – which is exactly what the append failed to give.

The rule worth keeping: never `>>` into a structured config file. The shell appends by file position; the format organises by section. They disagree. Same trap in YAML and INI. Open the file and place the line.

Part 4 - The two real bugs

These are the most valuable part of the session. Both were in working code that passed all 17 tests.

Bug 1 - 500 where 409 was correct

Symptom. Capturing an already-captured payment returned `500 Internal Server Error` with a long traceback, instead of `409 Conflict`.

The cause. In `capture()`, the operations were in this order:

```
payment = await _locked(session, payment_id)
clearing = await _clearing_account(session, payment.currency)
await ledger.post(...) # database write
await _transition(session, payment, PaymentStatus.CAPTURED) # state check
```

On a repeat capture, `ledger.post()` tried to insert a ledger transaction whose idempotency key already existed. The `uq_ledger_tx_idempotency` unique constraint rejected it, raising `IntegrityError` – **before the state machine was ever consulted**. `_mutate()` did not catch `IntegrityError`, so FastAPI fell through to its generic 500 handler.

What was actually working correctly. The database *did* prevent the double-spend. That is the second line of defence described in ADR 0002 – deliberately two independent guards, because double-spending is the failure you cannot apologise your way out of. The money was never at risk.

So why is this a bug at all? Because the *report* was wrong. A 500 tells the client “the server broke, retrying is reasonable”. The truth was “this already happened, stop”. A client obeying the 500 would retry indefinitely, generating load and log noise forever, and its user would never see a sensible message.

Correct behaviour with the wrong status code is still a bug. The status code is part of your API contract, not commentary on it. Something on the other end makes decisions based on it.

The fix. Check the state machine first:

```
payment = await _locked(session, payment_id)
assert_can_transition(payment.status, PaymentStatus.CAPTURED) # cheap, no I/O
clearing = await _clearing_account(session, payment.currency)
await ledger.post(...)
```

Plus an `IntegrityError` handler in `_mutate()` returning 409, as a backstop in case anything ever slips past the state machine.

The general principle: cheap checks before expensive ones. The state machine is a dictionary lookup in memory. It needed no database round trip and already knew the answer. Doing the expensive thing first and letting it fail wastes work *and* loses the specific error information – by the time `IntegrityError` surfaces, the reason “captured cannot go to captured” has been flattened into a generic constraint violation.

Why the placement is precise. The check sits *after* `_locked()` and *before* `_clearing_account()`. It must come after the lock, because you need the current status read under a lock to trust it – reading it unlocked means a concurrent request could change it between your read and your write. And it must come before any write, so nothing needs undoing.

Bug 2 - a demo that validated nothing

Symptom. After the first fix, the demo produced nonsense: two brand-new accounts were created, but the payment returned belonged to an *earlier* run, and both new accounts showed a balance of `0`. The first capture returned 409 when it should have returned 200.

The cause. `scripts/demo.sh` hardcoded `Idempotency-Key: demo-001`.

Idempotency was working **perfectly**. Same key, same payment returned, no duplicate created – exactly as designed. But

the script creates *fresh accounts* on every run while reusing a *fixed key*. So from the second run onward it silently operated on a payment belonging to accounts that no longer had anything to do with it.

Why this is the more dangerous of the two bugs. The first one announced itself with a 500 and a wall of traceback. This one produced plausible-looking output. Every line said `201 Created` and `200 OK`. Only by comparing the account UUID in the balance query against the one in the payment response could you see they did not match.

A test that passes for the wrong reason is worse than a failing test, because a failing test gets investigated.

The fix:

```
KEY="demo-$(date +%s) - $RANDOM"
```

`$(date +%s)` is the current Unix timestamp in seconds; `$RANDOM` is a random number. Together they make collisions essentially impossible.

Why not delete the old data before each run instead? That would mean the demo destroys data, which is a much worse property for a script you might one day run against something that matters. And the ledger is *append-only* by design – a script that deletes ledger rows contradicts the system’s central invariant. Generating a fresh key leaves all history intact, which is what an append-only system wants.

What these two bugs have in common

Both are about **state carried between invocations**.

- The ordering bug only appeared on the **second capture** of the same payment.
- The demo bug only appeared on the **second run** of the script.

And that is precisely why the 17 unit tests missed both. Those tests check functions in isolation with fresh inputs: the state machine correctly rejects `captured -> captured`, and `validate_postings` correctly rejects unbalanced entries. Both pass. **Neither test exercises the ordering between the state machine and the ledger**, because that ordering only exists when they are wired together with a real database.

That gap is exactly what integration tests are for, and it is the concrete argument for milestone 3’s integration test job being more than box-ticking.

When an interviewer asks “how do you test this?”, this is the answer with evidence behind it: unit tests for pure logic, integration tests for the interactions between components and for anything involving persisted state.

Part 5 - Consumer groups and rebalancing

The successful run produced logs worth understanding, because nobody configured this behaviour – it came free with the group ID.

```
Revoking previously assigned partitions ... partition=1
(Re-)joining group payments-workers
Elected group leader -- performing partition assignments using roundrobin
Setting newly assigned partitions {partition=2, partition=0, partition=1}
```

What happened. Earlier, two consumer processes were running in the same group (a leftover from a previous run). Kafka split the topic's three partitions between them – one took partition 1, the other took 0 and 2.

When one process died, Kafka noticed its **heartbeat** stopped arriving, declared it dead, and **rebalanced**: all three partitions were reassigned to the survivor. No messages were lost and nothing was processed twice.

Why this is the headline feature of consumer groups. Two properties fall out of it:

- **Scaling out** – start more copies of the worker with the same `group_id` and Kafka spreads partitions across them automatically. Throughput rises with no code change.
- **Fault tolerance** – lose a worker and its partitions are reassigned within seconds.

The ceiling: **you cannot usefully have more consumers in a group than there are partitions.** With three partitions, a fourth consumer sits idle. Partition count is therefore a capacity decision made when you create the topic – which is why `make topics` specifies `--partitions 3` rather than leaving it to a default.

The cost, which interviewers ask about. During a rebalance, consumption for that group **pauses**. On a large cluster with many consumers, frequent rebalances can cost more than the extra consumers gain. This is what `max_poll_interval_ms` in our consumer config guards against: if a handler takes longer than that window, Kafka assumes the consumer has died and triggers an *unnecessary* rebalance – which in the worst case loops forever, a failure mode called a rebalance storm.

Part 6 - Claude Code, and reviewing a diff

First real use this session.

The OAuth paste failure

```
OAuth error: Invalid code. Please make sure the full code was copied
```

Almost always a copy problem. The code is long, the terminal wraps it, and drag-selecting grabs a trailing newline or misses the tail. Triple-click to select the whole line, or use the page's copy button. Codes also expire in minutes.

Why approve edits one at a time

The approval prompt offered:

- ```
1. Yes
2. Yes, and switch to accept edits (auto-approve) for this session
3. No
```

**Option 1 was the right choice, and it will keep being the right choice.**

Option 2 disables review for everything that follows. For this project that is a bad trade, because the entire value is being able to explain the code in an interview. Approving a diff you have read is how the code enters your understanding at the same time it enters the repo. Auto-approve breaks that link, and you end up with commits authored by you that you cannot defend.

### What a good diff review looks like

For the ordering fix, three things were checked:

1. **Position** - is `assert_can_transition` after `_locked()` and before `_clearing_account()` ? It was. Placement was the whole point of the fix.
2. **Completeness** - was `void()` fixed too, with `VOIDED` rather than a copy-pasted `CAPTURED` ? It was.
3. **Reasoning recorded** - do the comments explain *why* the order matters, so the next person does not innocently "tidy" it back? They did.

Then a separate check that has nothing to do with the diff: **the tests passing did not prove the fix worked**. No test covers the ordering. The only real proof was re-running `make demo` and seeing a 409 with a clean message and no traceback.

A 409 *with* a traceback would have meant the wrong guard fired - the `IntegrityError` backstop rather than the state machine. Same status code, different mechanism, and only the log told them apart.

---

# Part 7 - Every command from this session

| Command                                                               | What it does                                          | Why this one                                                                                                                                                           |
|-----------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker compose logs kafka \\\ tail -40</code>                   | Last 40 lines of a container’s output                 | <code>tail</code> because logs are long and the fatal error is at the end. Do this <i>before</i> forming a theory.                                                     |
| <code>docker compose ps -a</code>                                     | All containers, including stopped, with exit codes    | Without <code>-a</code> , a crashed container is invisible – indistinguishable from never started.                                                                     |
| <code>docker stats --no-stream</code>                                 | Live CPU and memory per container                     | <code>--no-stream</code> prints once and exits; without it, it refreshes forever and holds the terminal.                                                               |
| <code>docker exec pp-postgres pg_isready -U app -d payments</code>    | Asks Postgres directly if it is accepting connections | Tests the database independently of the application, which separates “my code is wrong” from “the database is down”.                                                   |
| <code>docker compose up -d --force-recreate kafka</code>              | Rebuilds one container                                | Named service to avoid touching Postgres. <code>--force-recreate</code> because env vars are fixed at creation, so <code>restart</code> would reuse the broken config. |
| <code>grep -c "assert_can_transition" app/services/payments.py</code> | Counts matching lines                                 | <code>-c</code> counts instead of printing. A one-command check of whether an edit actually applied.                                                                   |
| <code>git commit --amend -m "..."</code>                              | Replaces the last commit                              | Fixes a bad message without adding a junk commit.                                                                                                                      |
| <code>git push --force-with-lease</code>                              | Pushes rewritten history safely                       | Refuses if the remote moved since you last looked. <code>--force</code> alone can destroy others’ work.                                                                |
| <code>kill %1 %2 %3</code>                                            | Stops backgrounded jobs by number                     | Ctrl+C might only reach one of three, leaving the others holding ports 8000 and 29092 and blocking the next run.                                                       |
| <code>make relay &amp; make worker &amp; make api &amp;</code>        | Three long-running processes in one terminal          | <code>&amp;</code> backgrounds each. They are servers – they never exit on their own.                                                                                  |

## Diagnostic sequence worth memorising

When a container will not work, in this order:

```
docker compose ps -a # is it even alive? what exit code?
docker compose logs <service> | tail -40 # what did it say before dying?
docker stats --no-stream # is memory the constraint?
```

Three commands, three different hypotheses, cheap to run. This is the shape of every debugging session: **narrow the cause before changing anything.**

# Part 8 - Where you actually are

**Milestone 2 complete and verified.** Pushed to `github.com/Aru-150905/payments-platform` at commit `cb602af`.

The passing demo proved, in one run:

- Idempotency: the same key returned the **same** payment, no duplicate created
- State machine: first capture `200`, second `409` with a clear message
- Double-entry: balances `-250000` and `+250000`, summing to exactly zero
- The full outbox chain: API wrote to Postgres, relay published 2 events to Kafka, consumer processed both with deduplication

**Two of seven milestones.** With tests, CI, four ADRs, and notes.

## Commits from this session

| Commit               | What                                             |
|----------------------|--------------------------------------------------|
| <code>6f8d811</code> | dev dependencies file                            |
| <code>7e489cd</code> | Kafka advertised-listener fix                    |
| <code>a927fd0</code> | pytest setting moved to the correct TOML section |
| <code>5954ac8</code> | state check before ledger write (the real bug)   |
| <code>cb602af</code> | unique idempotency key per demo run              |

## Next: milestone 3

Retries with exponential backoff, a dead-letter queue, a replay tool, and integration tests. The prompt is in `docs/SESSION-PROMPTS.md`.

Right now, a consumer handler that fails logs the error and drops the message. That is the honest current answer to “*what happens when a consumer cannot process a message?*” – which is the question that follows immediately after “why Kafka?” in any interview. M3 is the answer.

And after tonight you have the concrete justification for its integration tests, rather than a textbook one.

# Glossary - cumulative

---

Carrying forward session 01, with this session's additions.

**Advertised listener** – the address Kafka *tells clients to use*, as opposed to the address it binds to. Cannot be `0.0.0.0`.

**Alembic** – tool that applies versioned database schema changes.

**Alpine** – minimal Linux distribution; smaller, more secure images.

**Amend** – replace the most recent commit rather than adding another.

**Append-only** – never edited or deleted, only added to.

**At-least-once** – delivery where a message may arrive more than once but never zero times.

**Atomic** – happens completely or not at all.

**Backstop** – a secondary guard that catches what the primary one misses.

**Bottle** – Homebrew's prebuilt binary.

**Cask** – a Homebrew package that is a Mac application.

**CI** – automation that runs tests on every push.

**Clearing account** – holding account where money sits between authorization and capture.

**Consumer group** – consumers sharing an ID; they split partitions between them.

**Container** – isolated bundle of a program plus everything it needs.

**Deadlock** – two transactions each waiting for a lock the other holds.

**Dev dependency** – a package needed to develop or test, but not to run in production.

**DLQ** – dead letter queue; where messages go after retries are exhausted.

**Double-entry** – recording every movement twice, summing to zero.

**Effectively-once** – at-least-once delivery plus idempotent consumers.

**Ephemeral** – does not survive a restart.

**Exit code** – number a program reports on stopping. `0` success; `1` self-inflicted failure; `137` killed externally, usually out of memory.

**Force-recreate** – destroy and rebuild a container so new environment variables take effect.

**Healthcheck** – command a container runs repeatedly to prove it can actually serve requests, not merely that it is running.

**Heartbeat** – periodic signal a consumer sends so Kafka knows it is alive.

**Heredoc** – shell syntax feeding a block of text to a command, ending at a chosen terminator line.

**Idempotent** – doing it twice has the same effect as doing it once.

**Image** – read-only container template.

**Integration test** – test exercising several components wired together, usually with real infrastructure.

**KRaft** – Kafka's built-in coordination mode, replacing Zookeeper.

**Ledger** – append-only record of money movements.

**Linting** – automated checking for style problems and likely bugs.

**Lock** – claim on a database row so others must wait.

**Migration** – versioned, ordered schema change.

**Minor units** – smallest currency unit. Paise for INR.

**Nonroutable address** – an address traffic cannot be sent to. `0.0.0.0` is one.

**Offset** – a consumer's position in a partition.

**Outbox** – table where events are written in the same transaction as the business change, then published separately.

**Partition** – a slice of a topic. Ordering is guaranteed within one only.

**PATH** – folders the shell searches for commands.

**Pipe ( `|` )** – feeds one command's output into the next.

**Port** – numbered door on a machine.

**Rebalance** – Kafka reassigning partitions when group membership changes. Consumption pauses during it.

**Rebalance storm** – repeated unnecessary rebalances, usually caused by handlers slower than `max_poll_interval_ms`.

**Relay** – process that reads outbox rows and publishes them to Kafka.

**Shell** – program that reads and executes what you type. Yours is `zsh`.

**State machine** – explicit set of states and legal transitions.

**Tag** – version label on an image, after the colon.

**Topic** – a named Kafka stream.

**Transaction (database)** – group of changes that all succeed or all fail.

**Unit test** – test exercising one function in isolation, no infrastructure.

**Venv** – private folder with its own Python and packages.

**Volume** – Docker-managed storage that outlives the container. Deleted by `docker compose down -v`.

**Wheel** – precompiled Python package, specific to a Python version and platform.

**YAML** – text format where indentation defines structure.